The Journal of Open Source Software

Surrogates.jl: Surrogate Modeling and Surrogate-Based Optimization in Julia

Sathvik Bhagavan [1¶], **Ludovico Bessi**[4], **Vikramaditya Narayan**[5], **and Christopher Rackauckas** [2,3]

1 EPFL, Switzerland **2** Massachusetts Institute of Technology, USA **3** JuliaHub, USA **4** Google, Switzerland **5** Precogly, India ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 04 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Many scientific and engineering workflows depend on a function f that is expensive to evaluate: a PDE solve, a finite-element simulation, or a physical experiment. *Surrogate modeling* replaces f by a cheap approximation g fitted to a small design of experiments, and then optimizes, explores, or differentiates g instead.

`Surrogates.jl` gives a single Julia interface to a broad collection of surrogate families: radial basis functions (Buhmann, 2003), compactly supported Wendland kernels (Wendland, 1995), Kriging (Jones et al., 1998; Sacks et al., 1989) together with its gradient-enhanced (`GEK` (Han et al., 2013), `GEKPLS` (Bouhlel & Martins, 2019)) and partial-least-squares variants (`KPLS`, `KPLSK` (Bouhlel et al., 2016)), multivariate adaptive regression splines (Friedman, 1991), Lobachevsky splines (Allasia et al., 2013), inverse-distance (Shepard) interpolation (Shepard, 1968), linear and second-order polynomial models, and multi-fidelity correction surrogates (Forrester et al., 2007). Package extensions add mixtures of experts (Jordan & Jacobs, 1994), neural and gradient-enhanced neural networks (Bouhlel et al., 2020) via `Flux.jl` (Mike Innes, 2018), support vector machines via `LIBSVM` (Chang & Lin, 2011), gradient-boosted trees via `XGBoost` (Chen & Guestrin, 2016), polynomial chaos expansions via `PolyChaos.jl` (Mühlpfordt et al., 2020), and general Gaussian processes via `AbstractGPs.jl` (JuliaGaussianProcesses, n.d.).

Each fitted surrogate is a plain Julia function: `s(x)` evaluates the approximation and is differentiable with `ForwardDiff.jl` (Revels et al., 2016) and `Zygote.jl` (Michael Innes, 2018), `update!(s, x, y)` incorporates new observations without a full refit where the algorithm allows, and models with a predictive distribution (the Kriging family and Gaussian processes) implement `std_error_at_point`. The initial design is drawn with any sampler from `QuasiMonteCarlo.jl` (SciML, n.d.-a) (Sobol (Sobol', 1967), Latin hypercube (McKay et al., 1979), Halton (Halton, 1960), and others), plus a `SectionSample` strategy that holds chosen coordinates fixed.

Beyond fitting, `Surrogates.jl` implements surrogate-based optimization through `surrogate_optimize!`: stochastic RBF search (`SRBF`) (Regis & Shoemaker, 2007), DYCORS (Regis & Shoemaker, 2013), lower confidence bound (`LCBS`) (Cox & John, 1992; Srinivas et al., 2010) and expected improvement (`EI`) (Jones et al., 1998) acquisition, the SOP algorithm for parallel search centers (Krityakierne et al., 2016), and two multi-objective loops, surrogate-model-based screening (`SMB`) and the rolling tide evolutionary algorithm (Fieldsend & Everson, 2015) for noisy objectives. Where f can be evaluated at several points at once, an ask/tell entry point proposes a batch of candidates using constant-liar and Kriging-believer virtual points (Ginsbourger et al., 2010), for the SRBF, EI and LCBS strategies.

DRAFT

The Journal of Open Source Software

Statement of need

Surrogate modeling and surrogate-based optimization are used across aerospace design, hyperparameter tuning and simulation-based engineering, and mature toolkits exist outside Julia (Bouhlel et al., 2019; Eriksson et al., 2019; Saves et al., 2024). What they return is an isolated predictor, usable only from inside the toolkit. Enough for a standalone design study; not for the workflow that motivates this package.

Equation-based acausal modeling languages such as Modelica (Fritzson & Engelson, 1998) and, in Julia, `ModelingToolkit.jl` (Ma et al., 2021) assemble a system by composing component models and letting a compiler derive the equations. Replacing one expensive component with a surrogate is attractive exactly there, but only if the surrogate behaves like the rest of the model: callable from the generated code, and differentiable, because the solvers, parameter estimation and design optimization applied to the assembled system are gradient-based (Chris Rackauckas et al., 2021). A predictor reachable only through a foreign API is neither. Breadth alone has therefore not produced a mature surrogate ecosystem: the available packages each cover one family well while remaining unusable inside an equation-based model compiled elsewhere.

`Surrogates.jl` targets that setting within the Julia (Bezanson et al., 2017) and SciML (Christopher Rackauckas & Nie, 2017) ecosystems. Because a fitted surrogate obeys no protocol beyond being callable, it drops into Julia modeling and simulation code directly, and differentiation through a prediction needs no supporting code, a property the test suite checks for every family in both forward and reverse mode. Breadth matters here too: whichever family suits a component best, the same substitution must be available without learning a new API for each.

Functionality

A typical workflow has three stages:

1. **Sampling.** `sample(n, lb, ub, SobolSample())` builds the design of experiments with any `QuasiMonteCarlo.jl` sampler.
2. **Fitting.** Most surrogates follow `Surrogate(x, y, lb, ub; kwargs...)`, as in `Kriging(x, y, lb, ub)` or `RadialBasis(x, y, lb, ub)`, for scalar or vector inputs alike, and accept `update!` for new observations. The PLS and gradient-enhanced variants take extra positional arguments, as in `KPLS(x, y, n_comp, lb, ub, theta)` and `GEKPLS(x, y, grads, lb, ub)`. A subset (radial basis, linear, second-order polynomial, inverse distance, Lobachevsky, neural and `AbstractGPs`) supports vector-valued responses.
3. **Optimization.** `surrogate_optimize!(f, alg, lb, ub, surrogate, sampler)` drives the surrogate/expensive-function loop, evaluating `f` once per iteration. For parallel evaluation, `potential_optimal_points(alg, strategy, lb, ub, surrogate, sampler, n_parallel)` instead *asks* for a batch of `n_parallel` candidates and the caller *tells* the surrogate their responses with `update!`.

Several surrogates exploit extra problem structure to reach a given accuracy in fewer evaluations of f: `GEK` and `GEKPLS` fold in derivative information, often available for free from adjoint-based PDE solvers (Giles & Pierce, 2000), to raise accuracy per sample; `KPLS` and `KPLSK` fit correlation length scales in a PLS-reduced subspace to keep Kriging tractable in high input dimension; and `VariableFidelitySurrogate` corrects a cheap low-fidelity model with a small set of expensive high-fidelity samples, the common multi-fidelity setting in engineering design (Forrester et al., 2007).

DRAFT

State of the field

Table 1 places `Surrogates.jl` among comparable open-source toolkits. In Python, SMT covers the Kriging family with derivatives and mixed-variable support and provides EGO with EI, SBO and LCB infill criteria and qEI batching (Bouhlel et al., 2019; Saves et al., 2024); pySOT pairs RBF, GP, MARS and polynomial surrogates with the SRBF, DYCORS, SOP, EI and LCB strategies and asynchronous parallel evaluation (Eriksson et al., 2019); and BoTorch, with Ax, builds Monte-Carlo acquisition functions on PyTorch, including multi-objective batching (Balandat et al., 2020; Daulton et al., 2020). In R, DiceKriging and DiceOptim cover Kriging metamodeling and EGO with noisy, constrained and parallel variants (Picheny et al., 2025; Roustant et al., 2012), while mlrMBO makes the surrogate and infill criterion swappable over any `mlr` learner (Bischl et al., 2017). Dakota embeds surrogate-based optimization in a large simulation framework (Adams et al., 2020). Within Julia, `BayesianOptimization.jl` fits GP surrogates with EI, PI, UCB and Thompson acquisition (Brea, n.d.), while `GaussianProcesses.jl` (Fairbrother et al., 2022) and `AbstractGPs.jl` (JuliaGaussianProcesses, n.d.) supply GP models without an optimization loop.

Table 1: Comparison of open-source surrogate modeling toolkits.

Package	Language	Gradient-enhanced Kriging	Surrogate-based optimization	Multi-objective	AD through the surrogate
`Surrogates.jl` (this work)	Julia	GEK, GEKPLS	SRBF, DYCORS, SOP, EI, LCBS	SMB, RTEA	ForwardDiff, Zygote
SMT (Bouhlel et al., 2019; Saves et al., 2024)	Python	yes	EGO (EI, SBO, LCB), qEI batch	no	analytic derivatives
pySOT (Eriksson et al., 2019)	Python	no	SRBF, DYCORS, SOP, EI, LCB	no	no
BoTorch / Ax (Balandat et al., 2020)	Python	no	MC acquisition, batch	yes	PyTorch (Paszke et al., 2019)
DiceOptim (Picheny et al., 2025; Roustant et al., 2012)	R	no	EGO, noisy and parallel variants	no	no
`BayesianOptimization.jl` (Brea, n.d.)	Julia	no	EI, PI, UCB, Thompson	no	no

The contribution of `Surrogates.jl` is coverage rather than a new algorithm. Of the packages in Table 1, it is the only one that offers classical interpolation, the Kriging family with gradient enhancement and PLS dimension reduction, splines, and machine-learning surrogates behind one API, and the only one combining RBF-based global optimizers, Kriging-family acquisition functions and multi-objective surrogate loops in a single package.

We built a new package rather than contributing to an existing one because neither alternative reaches the same users. SMT and pySOT are Python libraries: calling them from Julia would

DRAFT

cross a language boundary on every evaluation of f and forfeit differentiation through the fitted surrogate. Within Julia, `GaussianProcesses.jl` and `AbstractGPs.jl` are Gaussian-process libraries by design, so the non-GP families and the optimization loops have no natural home there. Where shared infrastructure did make sense we contributed it rather than duplicating: sampling lives in `QuasiMonteCarlo.jl`, solvers in `Optimization.jl`, and the surrogate interface was extracted into `SurrogatesBase.jl` (SciML, n.d.-b) so that other packages can implement it without depending on this one.

Software design

Three decisions shape the package. First, the common interface is defined in a separate package, `SurrogatesBase.jl`, rather than here. A surrogate is either an `AbstractDeterministicSurrogate` or an `AbstractStochasticSurrogate`, and implements `update!`, `parameters` and `hyperparameters`; only the stochastic models add `std_error_at_point`. Splitting the interface from the implementations costs an extra package to maintain, but lets the optimization loops dispatch on capability rather than concrete type, and lets surrogates defined elsewhere participate.

Second, every family is implemented in Julia rather than wrapped around an external fitting engine. That is what makes the differentiability above possible, and the cost is carrying the implementations: the heavier backends (`Flux.jl`, `LIBSVM`, `XGBoost`, `PolyChaos.jl`, `AbstractGPs.jl` and the Gaussian-mixture expert model) are package extensions, loaded only when the user loads the backend. A default install therefore stays light while the surrogate list stays long.

Third, hyperparameter fitting is posed as an `OptimizationProblem` and handed to `Optimization.jl` (Dixit & Rackauckas, 2023) instead of carrying a bespoke optimizer: the Kriging family fits its correlation scales by maximum likelihood, searched with multi-start Nelder–Mead (Nelder & Mead, 1965) over a box of admissible scales. Parallel evaluation is exposed as the ask/tell pair above rather than as a callback, because the loop that dispatches expensive jobs to a cluster belongs to the user; the virtual points are placed on a private copy of the surrogate, so candidates within a batch repel each other without the caller's surrogate being touched.

Research impact statement

`Surrogates.jl` is used as a component by registered Julia packages in several fields: `ADRIA.jl`, a reef restoration decision-support platform from the Australian Institute of Marine Science; `AeroFuse.jl` (formerly `AeroMDAO.jl`), for aircraft design analysis; `Agtor.jl`, an agricultural systems model; `MaterialPointGenerator.jl`, for material point method landslide simulations; and `VariogramAnalysis.jl`, an implementation of the VARS global sensitivity analysis method (Razavi & Gupta, 2016). The repository has 383 stars and 78 forks, and has been the subject of several Google Summer of Code projects within the SciML organization.

AI usage disclosure

Generative AI was used in this project and is visible in its history. The Git log contains 46 commits authored under four AI-assisted identities and 53 commits carrying a `Co-Authored-By` trailer naming Claude; these cover refactoring, tests, documentation and bug fixes, all reviewed and merged through the normal pull-request process by human maintainers. This paper was drafted and revised with Anthropic's Claude via Claude Code; the authors verified every citation against publisher records, checked each claim against the source code, and are responsible for the content.

DRAFT

The Journal of Open Source Software

Acknowledgements

`Surrogates.jl` is developed as part of the SciML organization. Parts of the package were developed with support from Google Summer of Code, which funded several student contributors. We thank everyone who has contributed code, documentation, and bug reports through GitHub.

References

Adams, B. M., Bohnhoff, W. J., Dalbey, K. R., Ebeida, M. S., Eddy, J. P., Eldred, M. S., Hooper, R. W., Hough, P. D., Hu, K. T., Jakeman, J. D., Khalil, M., Maupin, K. A., Monschke, J. A., Ridgway, E. M., Rushdi, A. A., Seidl, D. T., Stephens, J. A., Swiler, L. P., & Winokur, J. G. (2020). *Dakota, a multilevel parallel object-oriented framework for design optimization, parameter estimation, uncertainty quantification, and sensitivity analysis: Version 6.13 user's manual* (No. SAND2020-12495). Sandia National Laboratories. https://doi.org/10.2172/1817318

Allasia, G., Cavoretto, R., & De Rossi, A. (2013). Lobachevsky spline functions and interpolation to scattered data. *Computational and Applied Mathematics*, *32*(1), 71–87. https://doi.org/10.1007/s40314-013-0011-0

Balandat, M., Karrer, B., Jiang, D. R., Daulton, S., Letham, B., Wilson, A. G., & Bakshy, E. (2020). BoTorch: A framework for efficient Monte-Carlo Bayesian optimization. *Advances in Neural Information Processing Systems*, *33*, 21524–21538.

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017). Julia: A fresh approach to numerical computing. *SIAM Review*, *59*(1), 65–98. https://doi.org/10.1137/141000671

Bischl, B., Richter, J., Bossek, J., Horn, D., Thomas, J., & Lang, M. (2017). *mlrMBO: A modular framework for model-based optimization of expensive black-box functions*. https://doi.org/10.48550/arXiv.1703.03373

Bouhlel, M. A., Bartoli, N., Otsmane, A., & Morlier, J. (2016). Improving kriging surrogates of high-dimensional design models by partial least squares dimension reduction. *Structural and Multidisciplinary Optimization*, *53*(5), 935–952. https://doi.org/10.1007/s00158-015-1395-9

Bouhlel, M. A., He, S., & Martins, J. R. R. A. (2020). Scalable gradient-enhanced artificial neural networks for airfoil shape design in the subsonic and transonic regimes. *Structural and Multidisciplinary Optimization*, *61*(4), 1363–1376. https://doi.org/10.1007/s00158-020-02488-5

Bouhlel, M. A., Hwang, J. T., Bartoli, N., Lafage, R., Morlier, J., & Martins, J. R. (2019). A python surrogate modeling framework with derivatives. *Advances in Engineering Software*, *135*, 102662. https://doi.org/10.1016/j.advengsoft.2019.03.005

Bouhlel, M. A., & Martins, J. R. R. A. (2019). Gradient-enhanced kriging for high-dimensional problems. *Engineering with Computers*, *35*(1), 157–173. https://doi.org/10.1007/s00366-018-0590-x

Brea, J. (n.d.). *BayesianOptimization.jl: Bayesian optimization for Julia*. https://github.com/jbrea/BayesianOptimization.jl.

Buhmann, M. D. (2003). *Radial basis functions: Theory and implementations*. Cambridge University Press. https://doi.org/10.1017/CBO9780511543241

Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, *2*(3), 1–27. https://doi.org/10.1145/1961189.1961199

Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings*

DRAFT

of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 785–794. https://doi.org/10.1145/2939672.2939785

Cox, D. D., & John, S. (1992). A statistical method for global optimization. *IEEE International Conference on Systems, Man, and Cybernetics*, 1241–1246. https://doi.org/10.1109/ICSMC.1992.271617

Daulton, S., Balandat, M., & Bakshy, E. (2020). Differentiable expected hypervolume improvement for parallel multi-objective Bayesian optimization. *Advances in Neural Information Processing Systems*, *33*, 9851–9864.

Dixit, V. K., & Rackauckas, C. (2023). *Optimization.jl: A unified optimization package*. https://github.com/SciML/Optimization.jl. https://doi.org/10.5281/zenodo.7738525

Eriksson, D., Bindel, D., & Shoemaker, C. A. (2019). *PySOT and POAP: An event-driven asynchronous framework for surrogate optimization*. https://doi.org/10.48550/arXiv.1908.00420

Fairbrother, J., Nemeth, C., Rischard, M., Brea, J., & Pinder, T. (2022). GaussianProcesses.jl: A nonparametric Bayes package for the Julia language. *Journal of Statistical Software*, *102*(1), 1–36. https://doi.org/10.18637/jss.v102.i01

Fieldsend, J. E., & Everson, R. M. (2015). The rolling tide evolutionary algorithm: A multiobjective optimizer for noisy optimization problems. *IEEE Transactions on Evolutionary Computation*, *19*(1), 103–117. https://doi.org/10.1109/TEVC.2014.2304415

Forrester, A. I. J., Sóbester, A., & Keane, A. J. (2007). Multi-fidelity optimization via surrogate modelling. *Proceedings of the Royal Society A*, *463*(2088), 3251–3269. https://doi.org/10.1098/rspa.2007.1900

Friedman, J. H. (1991). Multivariate adaptive regression splines. *The Annals of Statistics*, *19*(1), 1–67. https://doi.org/10.1214/aos/1176347963

Fritzson, P., & Engelson, V. (1998). Modelica — a unified object-oriented language for system modeling and simulation. In *ECOOP'98 — object-oriented programming* (Vol. 1445, pp. 67–90). Springer. https://doi.org/10.1007/BFb0054087

Giles, M. B., & Pierce, N. A. (2000). An introduction to the adjoint approach to design. *Flow, Turbulence and Combustion*, *65*(3–4), 393–415. https://doi.org/10.1023/A:1011430410075

Ginsbourger, D., Le Riche, R., & Carraro, L. (2010). Kriging is well-suited to parallelize optimization. In Y. Tenne & C.-K. Goh (Eds.), *Computational intelligence in expensive optimization problems* (pp. 131–162). Springer. https://doi.org/10.1007/978-3-642-10701-6_6

Halton, J. H. (1960). On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, *2*(1), 84–90. https://doi.org/10.1007/BF01386213

Han, Z.-H., Görtz, S., & Zimmermann, R. (2013). Improving variable-fidelity surrogate modeling via gradient-enhanced kriging and a generalized hybrid bridge function. *Aerospace Science and Technology*, *25*(1), 177–189. https://doi.org/10.1016/j.ast.2012.01.006

Innes, Michael. (2018). *Don't unroll adjoint: Differentiating SSA-form programs*. https://doi.org/10.48550/arXiv.1810.07951

Innes, Mike. (2018). Flux: Elegant machine learning with Julia. *Journal of Open Source Software*, *3*(25), 602. https://doi.org/10.21105/joss.00602

Jones, D. R., Schonlau, M., & Welch, W. J. (1998). Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, *13*(4), 455–492. https://doi.org/10.1023/A:1008306431147

DRAFT

Jordan, M. I., & Jacobs, R. A. (1994). Hierarchical mixtures of experts and the EM algorithm. *Neural Computation*, *6*(2), 181–214. https://doi.org/10.1162/neco.1994.6.2.181

JuliaGaussianProcesses. (n.d.). *JuliaGaussianProcesses/AbstractGPs.jl*. https://github.com/JuliaGaussianProcesses/AbstractGPs.jl.

Krityakierne, T., Akhtar, T., & Shoemaker, C. A. (2016). SOP: Parallel surrogate global optimization with pareto center selection for computationally expensive single objective problems. *Journal of Global Optimization*, *66*(3), 417–437. https://doi.org/10.1007/s10898-016-0407-7

Ma, Y., Gowda, S., Anantharaman, R., Laughman, C., Shah, V., & Rackauckas, C. (2021). *ModelingToolkit: A composable graph transformation system for equation-based modeling*. https://doi.org/10.48550/arXiv.2103.05244

McKay, M. D., Beckman, R. J., & Conover, W. J. (1979). Comparison of three methods for selecting values of input variables in the analysis of output from a computer code. *Technometrics*, *21*(2), 239–245. https://doi.org/10.1080/00401706.1979.10489755

Mühlpfordt, T., Zahn, F., Hagenmeyer, V., & Faulwasser, T. (2020). PolyChaos.jl – a Julia package for polynomial chaos in systems and control. *IFAC-PapersOnLine*, *53*(2), 7210–7216. https://doi.org/10.1016/j.ifacol.2020.12.552

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, *7*(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*.

Picheny, V., Ginsbourger, D., & Roustant, O. (2025). *DiceOptim: Kriging-based optimization for computer experiments*. https://doi.org/10.32614/CRAN.package.DiceOptim

Rackauckas, Chris, Anantharaman, R., Edelman, A., Gowda, S., Gwozdz, M., Jain, A., Laughman, C., Ma, Y., Martinuzzi, F., Pal, A., Rajput, U., Saba, E., & Shah, V. B. (2021). *Composing modeling and simulation with machine learning in Julia*. https://doi.org/10.48550/arXiv.2105.05946

Rackauckas, Christopher, & Nie, Q. (2017). DifferentialEquations.jl–a performant and feature-rich ecosystem for solving differential equations in julia. *Journal of Open Research Software*, *5*(1), 15. https://doi.org/10.5334/jors.151

Razavi, S., & Gupta, H. V. (2016). A new framework for comprehensive, robust, and efficient global sensitivity analysis: 1. theory. *Water Resources Research*, *52*(1), 423–439. https://doi.org/10.1002/2015WR017558

Regis, R. G., & Shoemaker, C. A. (2007). A stochastic radial basis function method for the global optimization of expensive functions. *INFORMS Journal on Computing*, *19*(4), 497–509. https://doi.org/10.1287/ijoc.1060.0182

Regis, R. G., & Shoemaker, C. A. (2013). Combining radial basis function surrogates and dynamic coordinate search in high-dimensional expensive black-box optimization. *Engineering Optimization*, *45*(5), 529–555. https://doi.org/10.1080/0305215X.2012.687731

Revels, J., Lubin, M., & Papamarkou, T. (2016). *Forward-mode automatic differentiation in Julia*. https://doi.org/10.48550/arXiv.1607.07892

Roustant, O., Ginsbourger, D., & Deville, Y. (2012). DiceKriging, DiceOptim: Two R packages for the analysis of computer experiments by kriging-based metamodeling and optimization.

DRAFT

Journal of Statistical Software, *51*(1), 1–55. https://doi.org/10.18637/jss.v051.i01

Sacks, J., Welch, W. J., Mitchell, T. J., & Wynn, H. P. (1989). Design and analysis of computer experiments. *Statistical Science*, *4*(4), 409–423. https://doi.org/10.1214/ss/1177012413

Saves, P., Lafage, R., Bartoli, N., Diouane, Y., Bussemaker, J., Lefebvre, T., Hwang, J. T., Morlier, J., & Martins, J. R. (2024). SMT 2.0: A surrogate modeling toolbox with a focus on hierarchical and mixed variables gaussian processes. *Advances in Engineering Software*, *188*, 103571. https://doi.org/10.1016/j.advengsoft.2023.103571

SciML. (n.d.-a). *QuasiMonteCarlo.jl*. https://github.com/SciML/QuasiMonteCarlo.jl.

SciML. (n.d.-b). *SurrogatesBase.jl*. https://github.com/SciML/SurrogatesBase.jl.

Shepard, D. (1968). A two-dimensional interpolation function for irregularly-spaced data. *Proceedings of the 1968 23rd ACM National Conference*, 517–524. https://doi.org/10.1145/800186.810616

Sobol', I. M. (1967). On the distribution of points in a cube and the approximate evaluation of integrals. *USSR Computational Mathematics and Mathematical Physics*, *7*(4), 86–112. https://doi.org/10.1016/0041-5553(67)90144-9

Srinivas, N., Krause, A., Kakade, S. M., & Seeger, M. (2010). Gaussian process optimization in the bandit setting: No regret and experimental design. *Proceedings of the 27th International Conference on Machine Learning (ICML)*, 1015–1022.

Wendland, H. (1995). Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. *Advances in Computational Mathematics*, *4*(1), 389–396. https://doi.org/10.1007/BF02123482

DRAFT